실전 웹서비스개발 – 12주차

서버리스 아키텍처 개념과 Supabase·Docker의 활용

서버리스 아키텍처

서버리스 아키텍처

- 서버리스 아키텍처의 등장
- 서버리스 아키텍처의 이해가 뭔지 한 문장으로 설명할 수 있게 되기
- BaaS vs FaaS 개념
- 서버리스의 장단점

서버리스 아키텍처의 등장 (1/3)

온프레미스(On-Premise)

- 회사가 직접 물리 서버를 사서, 전원·네트워크·냉방·보안까지 다 책임
- 데이터를 자체적으로 관리하므로 보안에 대한 통제 수준이 높음

개발자의 업무

- 서버 설치, OS 설치/업데이트
- 보안 패치, 백업, 로그 관리
- 트래픽 늘어나면 직접 서버 추가 구매

문제점

- 초기 비용 상승, 운영 인력 필요
- 서비스 만들기보다 서버 돌보는 시간이 더 많아짐

서버리스 아키텍처의 등장 (2/3)

클라우드 서비스

- AWS, Azure, GCP 같은 곳에서 가상 서버를 대여하는 서비스

IaaS (Infrastructure as a Service)

- 서버, 스토리지, 네트워크 같은 IT 인프라를 클라우드에서 빌려 사용하는 서비스

PaaS (Platform as a Service)

- 소프트웨어 개발에 필요한 플랫폼(운영체제, 데이터베이스, 개발 도구 등)을 클라우드에서 제공하는 서비스

SaaS (Software as a Service)

- 완전히 완성된 소프트웨어를 인터넷을 통해 제공하는 서비스

장점

- 하드웨어, 전원, 네트워크는 클라우드 업체가 관리
- 클릭 몇 번으로 서버 생성/삭제

단점

- OS 업데이트, 보안 설정, 서버 죽었는지 모니터링 우리가 해야 함
- 트래픽이 없을 때도 서버가 켜져 있으면 요금 계속 나감

서버리스 아키텍처의 등장 (3/3)

개발자가 서버와 DB를 관리하지 않고도 애플리케이션을 빌드하고 관리할 수 있는 소프트웨어 설계 방식

핵심 아이디어

- 서버 운영(관리)은 클라우드가, 비즈니스 로직 코드는 개발자가
- 요청이 있을 때만 잠깐 깨워서 코드 실행, 없으면 꺼둠
- 실행한 만큼만 돈을 내는 모델

이 모델이 왜 인기?

- 빠른 개발
- 높은 안정성
- 자동 확장
- 적은 초기 비용
- 스타트업, 사이드 프로젝트, 실험 서비스 같은 데서

서버리스 컴퓨팅

서버리스 컴퓨팅이란?

- 클라우드 제공자가 서버, 인프라, OS, 스케일링, 모니터링 등을 관리
- 개발자는 애플리케이션 코드만 작성해서 배포

특징

1. **서버 관리 없음**: OS, 패치, 용량 계획 신경 X
2. **자동 확장:** 트래픽에 맞춰 자동으로 늘었다 줄었다
3. **사용량 기반 과금:** 실행 시간/요청 수 기준
4. **이벤트 기반 실행:** HTTP 요청, 파일 업로드, DB 변경 등 이벤트 발생 시 코드 실행

https://aws.amazon.com/ko/what-is/serverless-computing/

FaaS (Function as a Service)

함수(코드 조각)를 그대로 클라우드에 올리고, 이벤트가 오면 그때만 실행

개발자가 하는 일

- handler 같은 작은 함수 단위로 로직 작성
- 클라우드에 업로드

동작 흐름

- 사용자가 버튼 클릭 → HTTP 요청 발생
- API Gateway 같은 서비스가 서버리스 함수(Lambda)를 호출
- 함수가 짧게 실행 → DB 읽고/쓰기 → 응답 반환
- 일정 시간 요청 없으면 함수 실행 환경은 사라짐 (비용 0에 가까움)

주요 서비스

- AWS Lambda, Azure Functions, Google Cloud Functions

BaaS (Backend as a Service)

백엔드에서 자주 쓰이는 기능들을 통째로 서비스로 제공하고, 개발자는 SDK나 API로 가져다 쓰기만 하는 백엔드 방식

개발자가 하는 일

- BaaS 콘솔에서 프로젝트 생성
- 클라이언트 앱(웹/모바일)에 SDK 설치 & 설정
- API 키, 프로젝트 ID 등 환경 변수 설정
- BaaS가 제공하는 기능 사용 ex) 회원가입/로그인(인증), DB 읽기/쓰기, 파일 업로드(스토리지)
- 보안 규칙 / 권한 설정
- 서버 코드를 따로 두지 않고, 프론트엔드에서 직접 BaaS API를 호출해 기능 구현

동작 흐름 (예: 회원가입 & DB 저장)

- 사용자가 React 앱에서 회원가입 폼 입력 → 버튼 클릭
- 프론트엔드에서 BaaS SDK의 signUp() 같은 함수를 호출
- BaaS의 인증 서버가 이메일/비밀번호 확인 후 새 유저를 생성 -> 토큰발급
- 프론트엔드는 이 토큰을 저장하고, 이후 DB 읽기/쓰기 요청 시 토큰을 함께 전송
- BaaS의 DB API가 권한/보안 규칙을 검사 후 요청에 맞게 데이터 읽기/쓰기를 수행
- 결과를 JSON 형태로 앱에 응답 → 화면 업데이트

주요 서비스

- Firebase, Supabase

서버리스 컴퓨팅 방식 비교

구분	BaaS	FaaS
개념	완성된 백엔드 기능을 API로 제공	내가 만든 함수를 이벤트 기반으로 실행
역할	인증, DB, 파일, 푸시 등 공통 기능 제공	비즈니스 로직(도메인 로직) 구현
장점	설정만 하면 빠르게 출시 가능	서버 없이 커스텀 로직 구현 가능
예시	Firebase, Supabase	AWS Lambda, Cloud Functions

BaaS vs FaaS 비교

서버리스 아키텍처 장점

서버 신경 X

- OS, 패치, 보안, 스케일링, 로드밸런서… → 클라우드가 알아서

개발 집중

- 비즈니스 로직 + UX에만 집중 가능

빠른 개발 & 릴리즈

- 작은 함수 단위로 쪼개서 배포 → 실험/수정이 쉬움

자동 확장 & 고가용성

- 트래픽이 갑자기 늘어도 자동으로 확장

비용 효율

- 사용량 기반 과금 → 학생/개인 프로젝트에 특히 유리

서버리스 아키텍처 단점

Cold Start

- 함수가 “잠에서 깰 때”까지 시간이 조금 걸려서 첫 요청이 느리게 응답할 수 있음

긴 작업에 부적합

- 함수당 실행 시간 제한 존재 (예: 몇 분 제한)
- 대용량 인코딩/학습 작업은 안 맞음 → 잘게 쪼개거나 다른 서비스로

Stateless 제약

- 함수는 이전 상태를 기억하지 않음 → 매 요청마다 초기화
- 상태가 필요하면 DB나 캐시 같은 외부 저장소를 써야 함

벤더 종속

- AWS 방식에 맞춰 만들면, GCP/Azure로 옮길 때 수정 양이 많을 수 있음
- 과금 정책·제한이 바뀌면 영향을 크게 받을 수 있음

서버리스를 쓰기 좋은 상황

트래픽이 들쭉날쭉한 서비스
- 이벤트 사이트, 캠페인 페이지, 시즌 이벤트용 API

스타트업 / MVP / 프로토타입
- 아이디어를 빨리 검증하고 싶은 초기 단계

간단한 API / 웹훅 처리
- 포털 로그인 후 콜백, 결제 완료 웹훅 처리 등

주기적/이벤트 기반 작업
- 매일 새벽 통계 집계, 업로드된 파일 처리, 알림 발송 등

서버리스를 쓰면 안되는 상황

지연에 매우 민감한 실시간 서비스

- 초저지연 게임 서버, 초당 수십/수백 요청이 상시 들어오는 환경

항상 CPU를 꽉 쓰는 장기 작업

- 영상 렌더링, ML 학습, 대규모 물리 시뮬레이션 등

플랫폼 종속을 피해야 하는 핵심 시스템

- 특정 클라우드 종속성이 비즈니스 리스크인 경우

Supabase

Supabase

PostgreSQL 기반의 오픈소스 BaaS(Backend as a Service)

- 데이터베이스
  - PostgreSQL(RDBMS)
- 기능
  - **Auth** – 이메일/소셜 로그인, 세션, JWT
  - **Storage** – 이미지/파일 업로드 및 관리
  - **Edge Functions** – 서버리스 함수(Deno 기반)
  - **Realtime** – DB 변경사항 실시간 구독
  - **자동 생성 API** – 각 테이블에 대한 REST/GraphQL API
- 다양한 언어의 SDK 지원
- 개발자는 서버 설치·운영 대신 스키마 설계 + 정책 설계 + 비즈니스 로직에 집중



PostgreSQL

- 확장 가능성 및 표준 준수를 강조하는 객체-관계형 데이터베이스 관리 시스템(ORDBMS)의 하나
- 발표일 1996년
- 현재 버전 18.1 / 2025년 11월 13일
- 저장소 : https://git.postgresql.org/gitweb/?p=postgresql.git
- 개발 언어 : C



Supabase에서 PostgreSQL 사용 이점

- 표준 SQL 사용(SELECT, INSERT, JOIN 등)
- 스키마, 트랜잭션 등 정통 RDBMS 개념 그대로 사용
- 나중에 직접 Postgres 서버를 운영해도 개념 재사용 가능
- Supabase를 배우면서 동시에 실제 RDB 설계 감각도 얻을 수 있음

Supabase 서버리스 아키텍처

인프라·서버 운영 없이 핵심 로직(RLS + DB 설계 + 함수)만 개발하면 되는 서버리스 백엔드

Supabase가 맡는 역할

- DB 서버(Postgres) 관리
- 인증(Auth): 로그인/로그아웃 & 세션
- 파일 서버(Storage): 이미지·파일 업로드
- 기본 API 자동 생성: 테이블 기준 REST/GraphQL
- 서버리스 실행환경(Edge Functions) 제공
- 인프라 운영, 스케일링, 보안 패치까지 포함

개발자가 할 일

- 테이블/스키마 설계 (Postgres 구조 만들기)
- 권한 정책(RLS) 설정: 누가 무엇을 볼 수 있는지 정의
- Edge Functions 작성: 필요한 비즈니스 로직만 구현

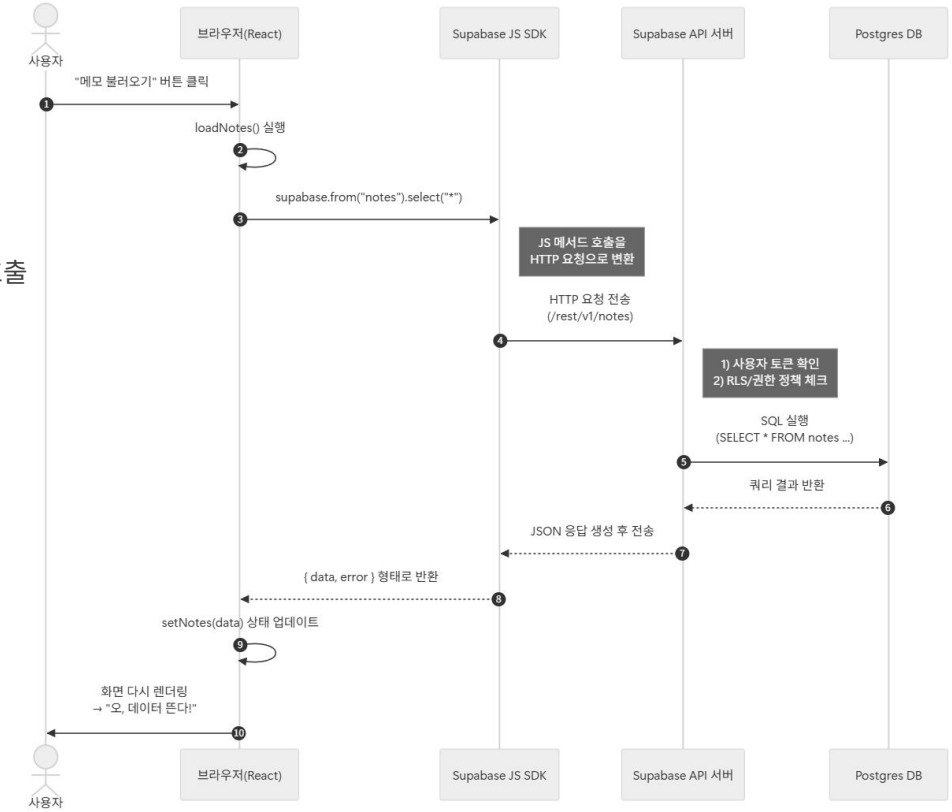
항목	전통 서버(Express+DB)	Supabase 서버리스
서버 코드	직접 라우트/컨트롤러 작성	대부분 필요 없음(함수만 작성)
DB 설치/관리	직접 설치/백업/보안	Supabase가 관리
인증/로그인	직접 구현 또는 라이브러리 조합	Auth 기능 내장
권한 제어	앱 코드 if문으로 처리	RLS 정책으로 처리
배포/스케일링	서버 인스턴스 직접 관리	Supabase 인프라 의존
자유도	매우 높음	플랫폼 제약 존재

전통 서버 개발과 Supabase 서버리스 개발 비교

기본 진행 흐름

ex) 서버에 저장된 노트 기록을 읽어올 경우

1) **사용자**
   - 브라우저(React) 화면에서 버튼 클릭
2) **React 코드**
   - loadNotes() 실행 → supabase.from('notes').select('*') 호출
3) **Supabase JS SDK**
   - 이 호출을 HTTP 요청으로 번역
   - Supabase API 서버로 전송
4) **Supabase API 서버**
   - 사용자 토큰 확인(누구인지)
   - RLS/권한 정책 체크
   - Postgres DB에 SQL 실행
   - 결과를 JSON으로 정리
5) **브라우저(React)**
   - Supabase JS가 JSON 응답을 받아 data로 넘겨줌
   - React 상태(setNotes(data))를 업데이트
   - 화면에 새로 렌더링



인증 과정

1) 로그인 과정

- 사용자가 이메일 + 비밀번호로 로그인 시도
- Supabase Auth가 이 정보를 DB의 사용자 계정과 비교
- 정보가 일치하면 → 로그인 성공

2) 로그인 성공 후 JWT 토큰 발급

- Supabase는 로그인에 성공한 사용자에게 JWT 토큰을 발급
- 토큰 정보(Bearer)
  - user id : 이 사용자를 구분하는 고유 ID
  - 토큰 만료 시간
  - 기타 권한 관련 정보: 역할(role), 발급시간(issued at), 프로젝트/정책 관련 metadata

3) 토큰을 보유 클라이언트

- React 앱(클라이언트)은 이 JWT 토큰을 브라우저 메모리/스토리지에 보관
- 이후 모든 Supabase 호출은 이 토큰을 함께 보내면서 실행

4) Supabase JS SDK의 역할

- Supabase JS SDK가 JWT 토큰을 자동으로 HTTP 헤더에 붙여서 Supabase API 서버로 요청을 전송
- 개발자는 단순한 JS 코드만 작성
- 인증 정보(토큰 처리)는 SDK가 자동 처리

권한(Authorization)

Row Level Security(RLS)

- 데이터 접근 권한을 서버 코드가 아닌 DB 자체가 책임지는 기능
- 어떤 사용자가 어떤 행(row)을 볼 수 있는지를 DB에서 직접 결정
- 각 행마다 접근 조건을 걸 수 있는 권한 시스템
    - user_id = auth.uid() 인 행만 조회/수정 가능
- **서버 코드를 많이 작성하지 않아도, DB 레벨에서 안전하게 권한을 제어**

프로젝트 생성

- Supabase 로그인 → New Project
- 입력 항목
  - Project name: ajou-pwd-demo 등
  - DB 비밀번호, Region
- 한 번의 생성으로
  - Postgres 서버 + Auth + Storage + Edge Functions 환경 동시 준비

프로젝트 전체가 우리의 백엔드 서버 인스턴스

대시보드

Table Editor

- Postgres 테이블을 시각적으로 만들고 컬럼을 관리하는 화면

SQL Editor

- 직접 SQL을 작성해 테이블 생성·수정·조회 등을 실행하는 콘솔

Database

- 테이블, 인덱스, 관계 등 전체 데이터베이스 구조를 관리하는 공간

Authentication

- 회원가입·로그인·소셜 로그인 등 사용자 계정을 관리하는 기능

Storage

- 이미지·동영상·파일을 업로드하고 저장하는 버킷(파일 서버)

Edge Functions

- 서버 없이 실행되는 서버리스 함수를 작성·배포·관리하는 공간

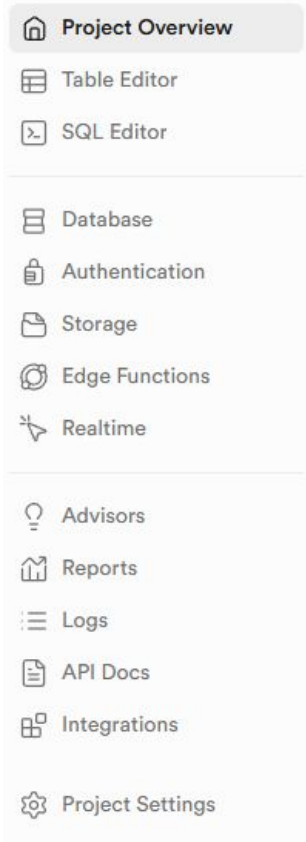


Table Editor

위치

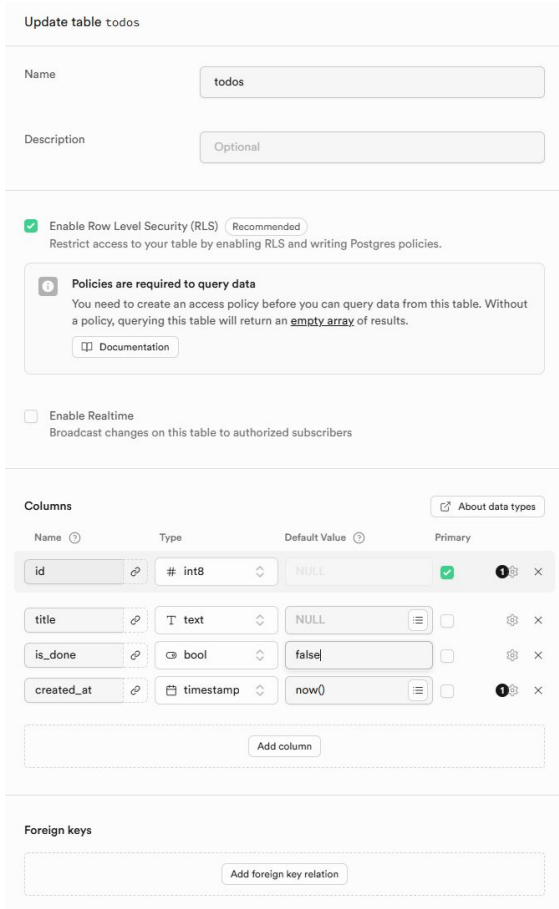
- Table Editor

역할

- Postgres 테이블(ERD) 설계 화면
- 컬럼 타입, PK/FK, 기본값 설정

예시

- todos 테이블
- id, title, is_done created_at 컬럼
- 나중에 from('todos')로 접근하는 대상 = 여기서 만든 테이블



SQL Editor

위치

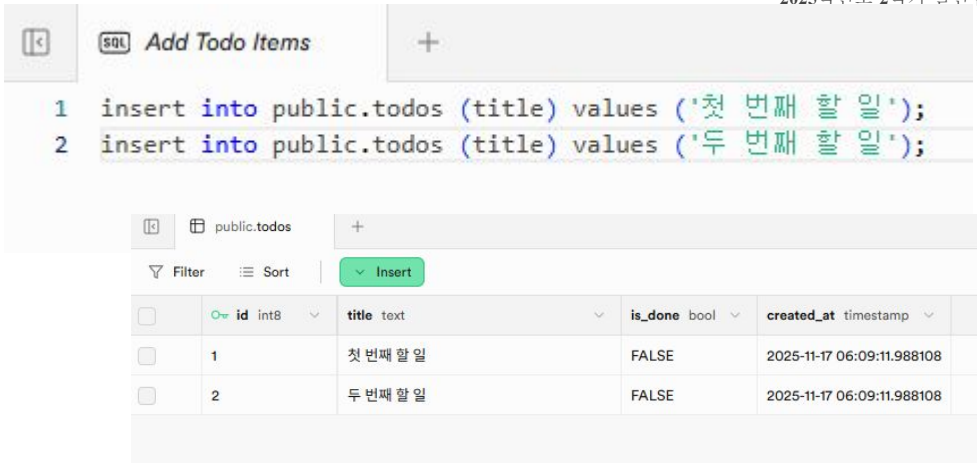
- SQL Editor

역할

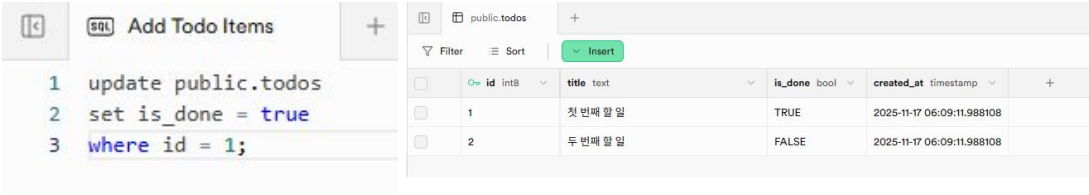
- 직접 SQL 작성 & 실행
- 테이블 생성, 수정, 데이터 조회/수정

중요한 점

- Supabase는 결국 Postgres에 SQL을 보내는 서비스
- ORM 없이도 SQL만으로 모든 작업 가능







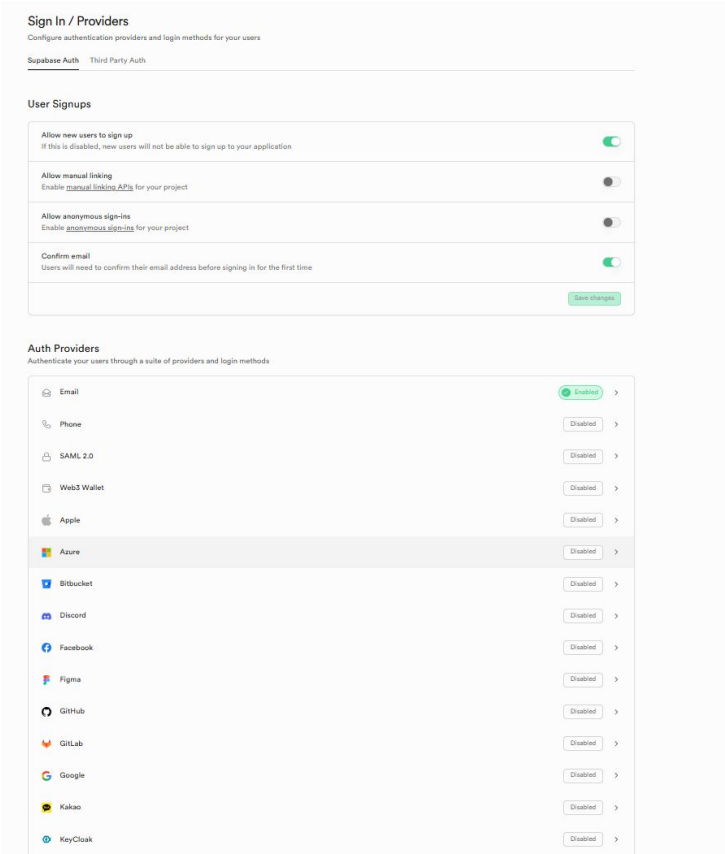
Authentication

역할

- 이메일/비밀번호, 소셜 로그인(Google, GitHub 등) 설정
- 가입된 사용자 목록(User 테이블) 확인

아키텍처 연결

- 로그인 성공 → 세션 & JWT 발급
- 이 토큰이 DB/RLS 접근 권한 판단의 기준



Policies / Row Level Security

위치

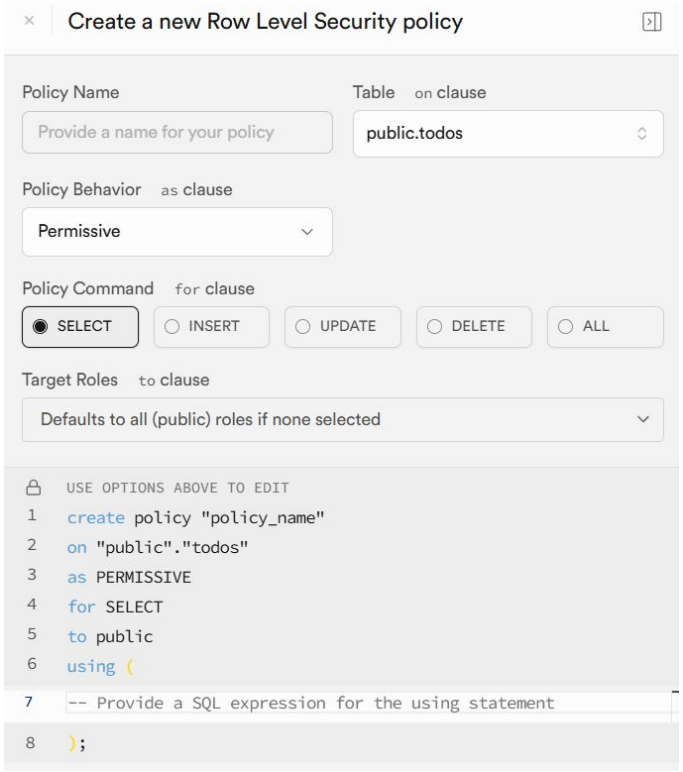
- Table Editor → 테이블 선택 → **RLS / Policies -> Create policy**

역할

- “각 행(row)을 어떤 사용자가 볼 수 있는지” 조건식으로 정의

의미

- 앱 코드가 아니라 **DB 정책**으로 권한 제어
- 클라이언트/Edge Function 어디서 접근하든 **일관된 보안 규칙** 유지



Storage – 파일 저장소

위치

- Storage

역할

- 이미지, 동영상, 파일을 저장하는 버킷(bucket) 관리

활용 예

- 프로필 이미지, 게시글 첨부 이미지, 게임/콘텐츠 리소스

아키텍처 포인트

- 단순 디스크가 아니라 업로드/다운로드 API + 권한/URL 관리까지 포함한 파일 서버

Edge Functions – 서버리스 함수 목록

위치

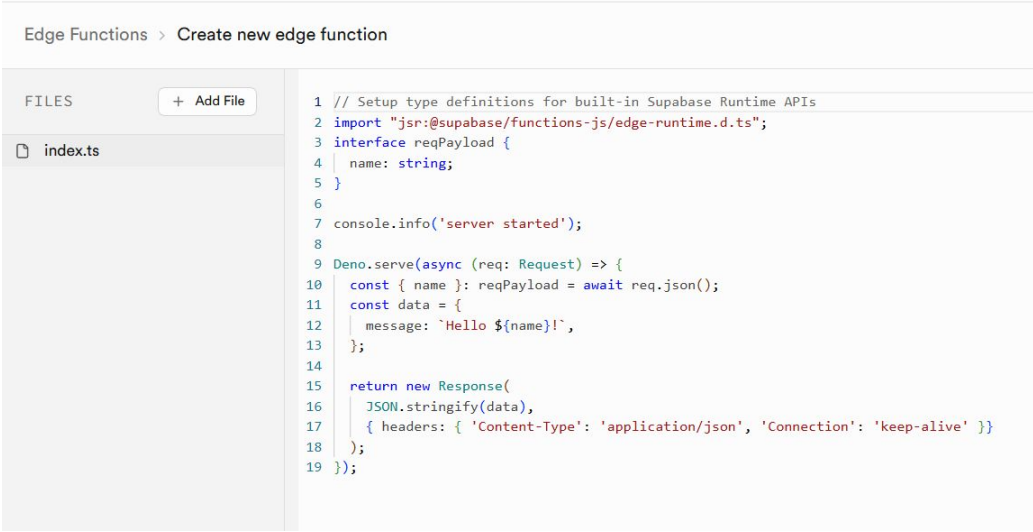
- Edge Functions

역할

- Supabase CLI로 배포한 서버리스 함수 목록 관리
- 각 함수 = HTTP 엔드포인트처럼 호출 가능

예시 활용

- 외부 API 호출(결제, 이메일 등)
- 복잡한 비즈니스 로직
- 서비스 role key를 써야 하는 민감한 작업 처리



Settings → API – 프로젝트 URL & 키

위치

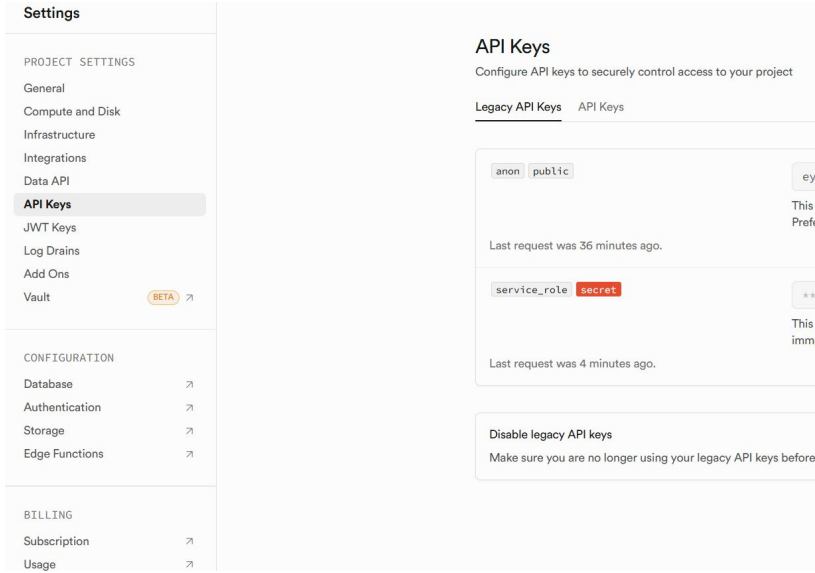
- **Settings → API Keys**

중요한 값

- **Project URL**: Supabase 인스턴스 기본 주소
- **anon public key:** 클라이언트에서 사용하는 공개 키
- **service role key:** 서버/Edge Function에서만 사용하는 강력한 키

클라이언트 호출

- createClient(Project URL, anon key)



클라이언트 직접 호출 패턴

클라이언트 직접 호출

- 구조: React → Supabase JS → DB/API 직접 호출
- 장점
  - 코드 단순, 빠르게 개발 가능
  - CRUD 중심 기능에 적합
- 주의점
  - anon key는 공개되므로, RLS 정책이 필수
  - RLS가 제대로 설정되지 않으면 데이터 과다 노출 위험

Edge Functions 패턴

- 구조: React → fetch('/functions/v1/my-api') → Edge Function → DB/외부 API
- 장점
  - 비공개 API 키, 복잡한 비즈니스 로직을 서버 쪽에 숨길 수 있음
  - Express 라우트와 비슷한 역할을 함수 단위로 수행

활용

직접 호출 패턴

- 단순한 CRUD, 읽기/쓰기 작업
- 예: 내 노트 목록, 게시글 리스트\

Edge Functions 패턴

- 보안/복잡도가 높은 작업
- 예: 결제, 외부 API 호출, 관리자용 배치 작업

실제 서비스에서는 두 방식을 혼합해서 사용

Supabase의 이해

Supabase는 결국 잘 관리된 PostgreSQL + 여러 백엔드 서비스 묶음

개발자가 집중할 부분

- 테이블/관계(ERD) 설계
- 인덱스, 제약조건 등 RDBMS 설계 원칙
- RLS 정책 설계

서버리스 = 서버가 없다는 뜻이 아니라, 서버 역할이 플랫폼과 DB 정책에 흡수되었다

Docker & Kubernetes

Docker

Docker(도커)는 컨테이너를 활용하여 소프트웨어를 배포 및 구동하기 위한 오픈 소스 플랫폼

등장 이유

과거의 문제:

- “내 컴퓨터에서는 되는데 서버에서는 안 된다”
- 개발 환경과 운영 환경의 차이
- OS 설정, 라이브러리 버전, Node/Python 버전 등이 충돌
- 서버 배포 시 반복되는 수작업(설치–구성–빌드–실행)

Docker는 이 모든 문제를 “환경 통일”로 해결하는 기술.

핵심 구성 요소

이미지(Image) = 실행 환경의 설계도

- 운영체제 + 런타임(Node, Python 등) + 라이브러리 + 앱 코드
- 변경 불가 (Immutable) → 항상 같은 결과 보장

컨테이너(Container) = 이미지의 실행본

- 가볍고 빠르게 실행
- 격리된 환경

Dockerfile = 이미지를 만드는 레시피

예:

1. FROM node:18
2. COPY . .
3. RUN npm install
4. CMD ["node", "index.js"]

동작 원리

Docker는 리눅스 커널 기능을 이용해 하나의 운영체제(OS) 위에 여러 개의 가상 실행 공간을 생성

Namespace

- 각각의 컨테이너가 파일시스템, 프로세스, 네트워크 등을 독립적으로 인식하도록 분리

Cgroups

- 컨테이너에게 CPU, 메모리 등 자원 제한

유저 공간 공유

- VM과 달리 OS 자체를 복제하지 않음 → 매우 가벼움

컨테이너는 '가벼운 가상환경'을 만드는 OS 레벨 기술.

Kubernetes

컨테이너화된 애플리케이션의 배포, 확장, 관리를 자동화하는 오픈소스 플랫폼

- Google이 Borg 시스템을 일반화

등장 이유

- Docker는 개별 서버에서는 좋지만 서비스 규모가 커지면 유지하기 어려운 문제 발생
  - 컨테이너가 갑자기 죽으면? → 자동 복구 필요
  - 트래픽 증가하면? → 자동 확장 필요
  - 서버 여러 대에 분산해야 하면?
  - 지속적인 배포(Zero-Downtime Deployment) 필요
  - 설정/상태를 일관성 있게 유지해야 함



Docker와 Kubernetes 관계

역할	Docker	Kubernetes
목적	컨테이너 생성·실행	수백/수천 개 컨테이너 운영 자동화
단위	이미지, 컨테이너	Pod, Deployment, Service
장점	가벼움, 이식성	자동 복구, 자동 확장, 배포 전략
운영 방식	개별 서버 단위	클러스터 전체 단위
스코프	개발 환경 구성	운영 환경 구축

요약

Serverless

- 운영을 숨김 (Supabase, Firebase)

Docker

- 운영을 드러냄 (환경을 직접 구성)

Kubernetes

- 운영을 자동화 (스케일링/배포/복구)

Supabase CLI 이용한 Docker 개발환경 구축

Docker 설치

Docker Desktop

- **Windows**
  - WSL 2 설치
  - PowerShell을 관리자 권한으로 실행 후

```
wsl --install
```

- **Docker Desktop 다운로드 및 설치**
  - https://www.docker.com/get-started/
  - "Download for Windows" 클릭
  - 설치 파일 실행
  - 설치 완료 후 재부팅
  - Docker 설치 확인

```
docker --version
docker compose version
```



Supabase 프로젝트 생성

프로젝트 폴더 생성

```
mkdir my-supabase-project
cd my-supabase-project
```

Supabase 초기화

```
npx supabase init
```

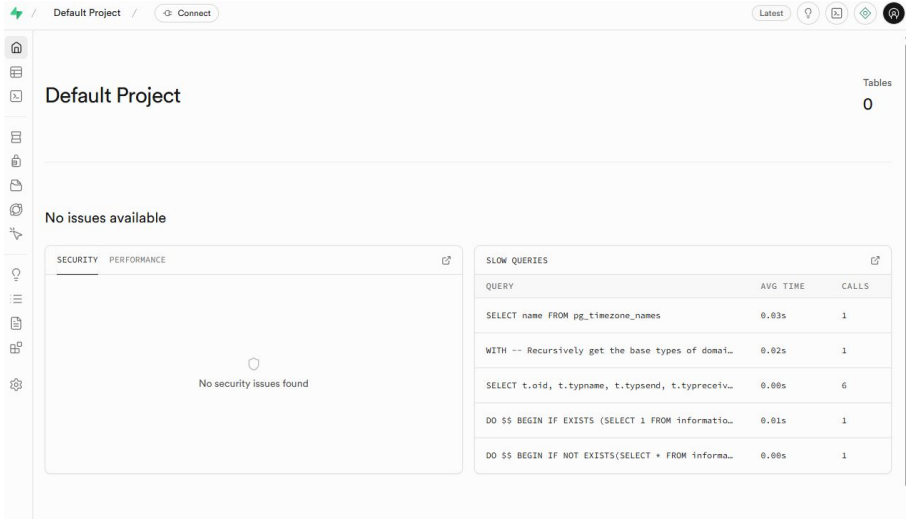
로컬 Supabase 시작과 종료

```
npx supabase start
```

```
npx supabase stop
```

```
Digest: sha256:db1fb41e13ebd8666116486a9741bf1a1cb1cba36b4d0e44f45d95fca70576ac
Status: Downloaded newer image for public.ecr.aws/supabase/studio:2025.11.10-sha-5291fe3
Waiting for health checks...
Started supabase local development setup.

         API URL: http://127.0.0.1:54321
     GraphQL URL: http://127.0.0.1:54321/graphql/v1
  S3 Storage URL: http://127.0.0.1:54321/storage/v1/s3
         MCP URL: http://127.0.0.1:54321/mcp
    Database URL: postgresql://postgres:postgres@127.0.0.1:54322/postgres
      Studio URL: http://127.0.0.1:54323
     Mailpit URL: http://127.0.0.1:54324
 Publishable key: sb_publishable_ACJWlzQHlZjBrEguHvfOxg_3BJgxAaH
      Secret key: sb_secret_N7UND0UgjKTVK-Uodkm0Hg_xSvEMPvz
   S3 Access Key: 625729a08b95bf1b7ff351a663f3a23c
   S3 Secret Key: 850181e4652dd023b7a98c58ae0d2d34bd487ee0cc3254aed6eda37307425907
       S3 Region: local
```



Supabase CLI 명령어

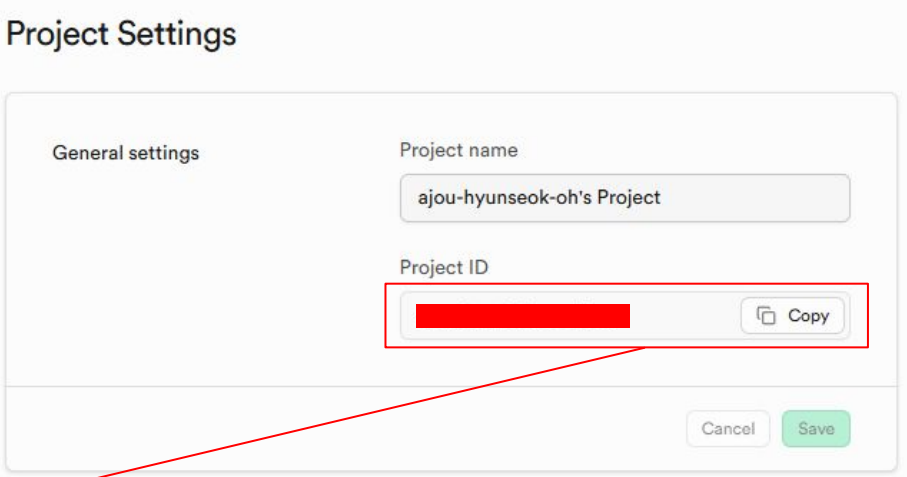
명령어	설명
npx supabase start	로컬 서버 시작
npx supabase stop	로컬 서버 중지
npx supabase status	현재 상태 및 접속 정보 확인
npx supabase db reset	데이터베이스 초기화
npx supabase migration new 이름	새 마이그레이션 생성
npx supabase db push	마이그레이션 적용

로컬-원격 프로젝트 연동

원격 프로젝트와 연결

1. https://supabase.com/dashboard 접속
2. 연결할 프로젝트 선택
3. Settings → General settings
4. Project ID 값 복사
5. Supabase 계정 로그인

```
npx supabase login
```



프로젝트 링크

```
npx supabase link --project-ref [Project ID]
```

- 데이터베이스 비밀번호 입력 (프로젝트 생성 시 설정한 비밀번호)

프로젝트 링크

```
npx supabase unlink
```

마이그레이션

- 마이그레이션은 데이터를 한 컴퓨팅 환경이나 스토리지 시스템에서 다른 환경으로 이동
- 데이터베이스 구조의 변경사항을 관리
- 마이그레이션 네이밍 컨벤션
  - YYYYMMDDHHMMSS_동사_명사_테이블.sql

```
# 현재 적용된 마이그레이션 확인
npx supabase migration list

# 로컬과 원격 DB 차이 확인
npx supabase db diff

# 마이그레이션 실행 계획 미리보기
npx supabase db push --dry-run

# 특정 시점으로 롤백
npx supabase db reset --version 20241123143022

# 마이그레이션 히스토리 조회
SELECT * FROM supabase_migrations ORDER BY
executed_at DESC;
```

```
-- 001_create_users_table.sql
-- 설명: 사용자 정보를 저장하는 테이블 생성

CREATE TABLE IF NOT EXISTS users (
    id BIGSERIAL PRIMARY KEY,
    email TEXT UNIQUE NOT NULL,
    name TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

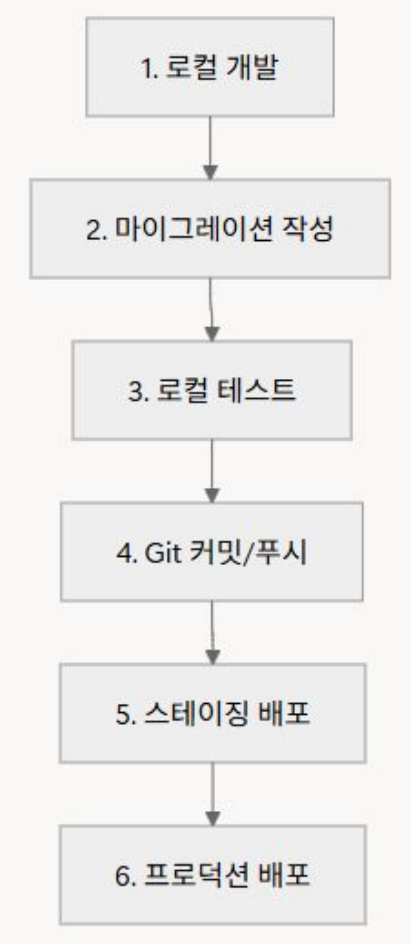
-- 인덱스 추가 (이메일로 빠른 검색)
CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);
```

```
# 1. 로컬에서 마이그레이션 생성
npx supabase migration new create_users_table

# 2. SQL 작성 (supabase/migrations/xxx.sql)

# 3. 로컬 테스트
npx supabase db reset
npx supabase start

# Studio에서 확인
```



로컬-원격 배포

스키마 배포

```
npx supabase db push # 로컬
```

```
npx supabase db push --linked # 원격
```

스키마를 로컬에 동기화

```
npx supabase db pull # 원격에서 스키마 가져오기
npx supabase db reset # 로컬 DB에 적용
```

프로젝트 연결 해제

```
npx supabase unlink
```

트러블슈팅

Docker 관련 오류

- Docker Desktop이 실행 중인지 확인
- WSL 2가 제대로 설치되었는지 확인: wsl --status

포트 충돌

- 다른 프로그램이 54321-54323 포트를 사용 중이면 충돌 발생.

```
# 54322 포트를 publish한 컨테이너 모두 중지 + 삭제
docker ps -a --filter "publish=54322" -q | ForEach-Object { docker stop $_; docker rm $_ }

# 이름에 supabase가 들어간 컨테이너 모두 중지 + 삭제
docker ps -a --filter "name=supabase" -q | ForEach-Object { docker stop $_; docker rm $_ }

npx supabase stop   # 혹시 떠 있는 게 있으면 한 번 더 정리
npx supabase start
```

Q&A